Day 158: Implicit Feedback and Cold Start: Learning from What People Watch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 5 | Complete study guide

Watch: Build a Spotify-Like Music Recommender System in Python (Valerio Velardo - The Sound of AI) | Mitigating the challenges of cold start in TensorFlow Recommenders (TensorFlow)

Script: `day158_implicit_feedback.py` · every number in this guide is copied by program from `day158_run_output.txt`, the real output of that script.

Most real systems never see star ratings, only who watched, clicked or bought what. Today MovieLens's ratings become watched / not watched, and the question becomes “what will this user watch next?”. Implicit ALS, which treats every empty cell as a weak 0, put 3.67 of every 10 recommended films among the ones the user went on to watch (0.3672), against 0.1948 for popularity. The same lists also hit far more *liked* films (0.2653) than Day 157's best rating model (0.0684). For a brand-new user, a single solve from the films they had watched (fold-in) took recall@10 from 0.0925 (popularity) to 0.1584 once 20 films were known; their age and gender barely helped.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

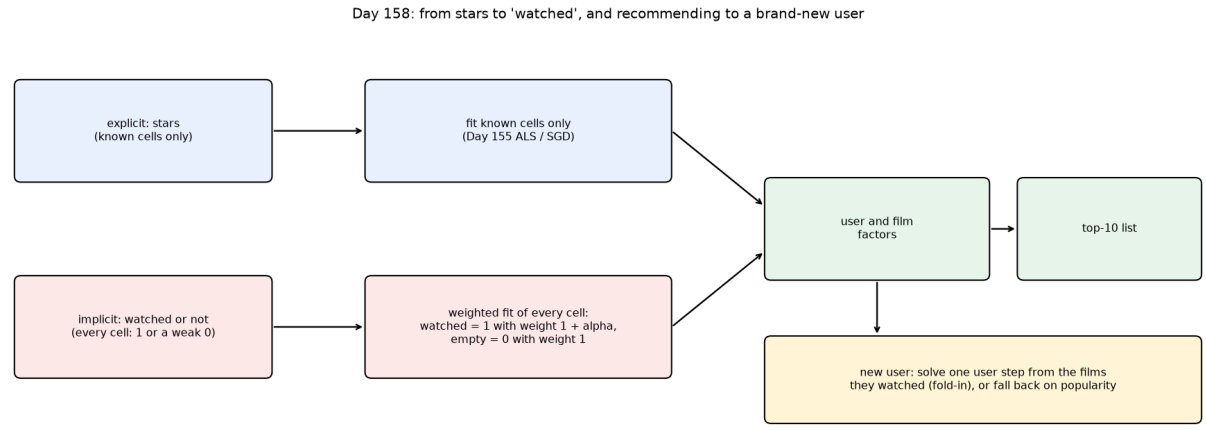
Explicit feedback is what a user tells you on purpose, such as a star rating. **Implicit feedback** is what they do: watch, click, buy, listen, skip. It is plentiful, but it has no negatives: an empty cell may mean “did not like” or simply “never saw it”.

The **cold-start problem** is recommending when there is no history. Day 156 handled new *films* with their content; today handles new *users*.

REAL-WORLD SCENARIO

A music app knows that you played a song 40 times and skipped another after 5 seconds, but you never rated either. It has to learn your taste from what you did, and for a new sign-up it has only the first few songs you choose.

SYSTEM DIAGRAM



Explicit stars fit only the known cells; implicit data fits every cell, with more weight on what was watched. The film factors also serve brand-new users.

Today's part for the week's build

`day158_exercise.py` is today's parts (load_users, implicit_als, implicit_objective, fold_in, cooccurrence and topk_metrics), checked by hand.

ACTUAL OUTPUT

```
load_users: 943 users, ages 7-73, 273 women, 21 occupations
objective, one user and two films, x = 1, y = (1, 0.5), confidence 3 on film 0, penalty 0.1:
  watched cell 3 x (1 - 1)^2 = 0; empty cell 1 x (0 - 0.5)^2 = 0.25; penalty 0.1 x (1 + 1 + 0.25)
      = 0.225; total 0.475; implicit_objective gives 0.4750
cosine: films 0 and 1 share 1 watcher of 2 and 1, so 1 / sqrt(2 x 1) = 0.7071; films 0 and 2 share
    none: cooccurrence gives 0.7071 and 0.0000
top-2 of scores 9, 5, 4, 1 with film 0 seen: films 1 and 2; held out 1 and 3, so 1 hit: precision
    0.5, recall 0.5; topk_metrics gives 0.50, 0.50, list [1, 2]
fold_in against the full weighted ridge over every film: largest difference 0.0e+00
  the same check can fail: leaving out the extra weight c - 1 misses by 0.899
implicit_als on 12 x 8: objective after each sweep 124.812, 40.294, 36.471, 34.708, 33.606,
    32.841; never rises: True
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. From stars to ‘watched’: the data, the measure, and popularity

DEFINITION — start here, no prior knowledge assumed

Every rating on Day 154's 80,000-rating training split becomes a 1 (“watched”) whatever its stars; every other cell is 0. The 20,000 test ratings become the films each user is going to watch.

Precision@10: of the 10 films recommended (never one already watched), the share the user watched in the test set. **Recall@10**: of the films the user watched in the test set, the share that made the top 10. Both are averaged over users.

REAL-WORLD SCENARIO

A bookshop sees what you bought, not whether you enjoyed it. A book you bought and hated still counts as a purchase.

MATHEMATICS

$$\textit{precision@k} = \textit{hits} / k; \quad \textit{recall@k} = \textit{hits} / (\textit{held-out films of the user}); \quad \textit{hits} = \textit{held-out films in the top } k$$

IMPLEMENTATION

```
def topk_metrics(S, seen, held, k=10):
    S = np.where(seen, -np.inf, S); who = np.flatnonzero(held.sum(1) > 0)
    top = np.argsort(-S, axis=1, kind="stable")[:, :k]
    h = np.take_along_axis(held, top, 1)[who].sum(1)
    return h.mean() / k, (h / held[who].sum(1)).mean(), top[who]
```

ACTUAL OUTPUT

```
training: 80000 watched cells of 1586126 (5.0%); test: 20000 held-out watched films for 942 users
every rated film counts as watched, whatever its stars: 17.6% of test 'watched' films got 1-2 stars
precision@10 / recall@10 on held-out watched films: popularity 0.1948 / 0.1195; random 0.0151 / 0.0064
Day 154's measure (test films rated 4-5): popularity 0.1381
Actual output from this lesson's run.
```

What it means

Only 5.0% of cells are watched. Turning ratings into “watched” throws away information: 17.6% of the test films counted as watched were rated 1 or 2 stars.

On this measure popularity scores 0.1948 (random scoring 0.0151); on Day 154's measure, which counts only test films rated 4–5, the same lists score 0.1381. Both measures are reported below.

WHY CHOOSE THIS?

Implicit data is everywhere and costs the user nothing; the measure asks what a recommender really has to do: predict what will be chosen.

WHY NOT THIS? (TRADEOFFS)

A watch is not a like, and an empty cell is not a dislike; the stars carried information that is now ignored.

2. Implicit ALS: every empty cell is a weak 0

DEFINITION — start here, no prior knowledge assumed

Hu, Koren and Volinsky (2008) fit *every* cell of the table: 1 for watched, 0 for not watched. Each cell gets a **confidence** weight: 1 + α where watched (we are fairly sure the user is interested), 1 elsewhere (a weak guess that they are not). ALS alternates: solve every user's factors with the films' fixed, then every film's.

Fitting all 1.6 million cells sounds slow, but the empty cells all have weight 1, so their part, Y^TY, is computed once per half-sweep and shared by every user; each user then adds a correction only for the films they watched.

REAL-WORLD SCENARIO

A teacher who marks every unanswered question as “probably does not know”, but trusts the answered questions far more.

MATHEMATICS

$$\text{minimise } \Sigma_{\text{all } u,\, i}\, c_{ui}\,(p_{ui} - x_u \cdot y_i)^2 + \lambda\,(\Sigma\,|x_u|^2 + \Sigma\,|y_i|^2); \quad p = 1 \text{ if watched else } 0; \quad c = 1 + \alpha \text{ if watched else } 1$$

$$\text{user step: } x_u = (Y^TY + Y_w{}^T\, \mathrm{diag}(c - 1)\, Y_w + \lambda I)^{-1}\, Y_w{}^T\, c_w \quad (w = \text{the films the user watched})$$

IMPLEMENTATION

```
def implicit_als(C, k, lam, sweeps=10, seed=0, track=None):
    nu, ni = C.shape; rng = np.random.RandomState(seed)
    X, Y = 0.01 * rng.randn(nu, k), 0.01 * rng.randn(ni, k)
    hist = []
    for _ in range(sweeps):
        for A, B, Cm in ((X, Y, C), (Y, X, C.T)):
            G = B.T @ B + lam * np.eye(k)
            for a in range(A.shape[0]):
                j = np.flatnonzero(Cm[a]); c = Cm[a, j]
                Bj = B[j]
                A[a] = np.linalg.solve(G + (Bj * (c - 1)[:, None]).T @ Bj, Bj.T @ c)
        hist.append(track(X, Y) if track else ())
    return X, Y, hist
```

SPECIFICATION

Data	Day 154's split; the grid is scored on its validation part (10% of training), the winner refitted on all of training
Grid	factors 10, 20, 50, 100 × penalty λ 1 to 300 × α 1, 3, 10, 30; 10 sweeps each; chosen on validation precision@10
Check	the shortcut user step against the plain weighted ridge over every cell

ACTUAL OUTPUT

```
check: the shortcut user step against the full weighted ridge over every cell: largest difference 0.0e+00
grid: 4 factor counts x 6 penalties x 4 confidence weights (578 s); validation precision@10 at 50 factors:
   penalty   alpha 1   alpha 3  alpha 10  alpha 30
         1    0.1128    0.1093    0.1010    0.0905
         3    0.1158    0.1118    0.1027    0.0910
        10    0.1289    0.1192    0.1097    0.0948
        30    0.1284    0.1374    0.1216    0.1008
       100    0.0708    0.1009    0.1233    0.1279
       300    0.0706    0.0704    0.0840    0.1056
  best at each factor count: 10: 0.1333, 20: 0.1363, 50: 0.1374, 100: 0.1337
chosen: 50 factors, penalty 30, alpha 3 (validation 0.1374)
```

What it means

The shortcut and the full weighted ridge agree exactly here (largest difference 0.0e+00); the exercise shows the same check missing by 0.899 when the extra weight c – 1 is left out, so it can fail.

Validation chose 50 factors, penalty 30, alpha 3. Penalty and α interact: a large penalty needs a large α (penalty 100 does best with α 30, 0.1279), and with penalty 300 and α 1 the model collapses to 0.0706, about half the best. **First draft:** the first run tried only 10, 20 and 50 factors and chose 50, the edge of the grid, so 100 was added; it scored 0.1337, below 50's 0.1374.

The objective fell on every one of the 10 sweeps, as ALS guarantees, since each step solves its half exactly:

ACTUAL OUTPUT

```
objective after each sweep: 303612, 167523, 160481, 158447, ..., 156536; never rises: True
```

WHY CHOOSE THIS?

Uses every cell, not just the watched ones, so the model learns what users do *not* pick; exact steps, no learning rate.

WHY NOT THIS? (TRADEOFFS)

λ, α and the factor count must be tuned together, and a full grid is slow; the weak-0 assumption is wrong for films the user simply never saw.

3. Who wins on the top-10 lists?

DEFINITION — start here, no prior knowledge assumed

Six models on the same test: **popularity**; **co-occurrence** (item-item cosine on who watched what, a film scored by its similarity to the user's films); **PureSVD** (the best rank-k fit of the 0/1 table, rank chosen on validation); **Day 155's explicit ALS** (stars predicted from rated cells only); **implicit ALS**; and a **control**: implicit ALS with users' factors shuffled between users.

'Median watchers' is how many training users watched the typical recommended film: how mainstream the lists are.

REAL-WORLD SCENARIO

Judging forecasters on rain they predicted, not on how precisely they guessed the temperature.

IMPLEMENTATION

```
def cooccurrence(B):
    n = np.sqrt(B.sum(0))
    S = (B.T @ B) / np.maximum(np.outer(n, n), 1e-12)
    np.fill_diagonal(S, 0.0)
    return S
```

ACTUAL OUTPUT

```
PureSVD rank chosen on validation: 20 (5: 0.1142, 10: 0.1281, 20: 0.1313, 50: 0.1125, 100: 0.0852)
                                  test  precision@10  recall@10  liked: precision@10  median watchers
                            popularity        0.1948     0.1195               0.1381              346
              co-occurrence (item-item)        0.2930     0.1948               0.2140              247
                        PureSVD, rank 20        0.3507     0.2301               0.2479              216
           explicit ALS (Day 155, stars)        0.0421     0.0189               0.0405                7
                 implicit ALS (50, 30, 3)        0.3672     0.2438               0.2653              233
   control: implicit ALS, users shuffled        0.1288     0.0723               0.0909              242
Actual output from this lesson's run.
```

What it means

Implicit ALS was best on every column: 0.3672 of its top-10 films were watched and 0.2653 were liked (rated 4–5), against popularity's 0.1948 and 0.1381. PureSVD (0.3507) and co-occurrence (0.2930) also beat popularity by a wide margin. The shuffled control fell to 0.1288, below popularity: the gain comes from each user's own factors.

This answers the week's puzzle. Day 155's explicit ALS, trained only on rated cells, scored 0.0421, and its films had a median of just 7 watchers. A model of *how many stars* never learns *which films a user picks*; a model trained on every cell does. Its liked-film precision is nearly four times Day 157's best rating model (0.0684).

WHY CHOOSE THIS?

Built for the task a list actually has: choosing a few films out of 1,682.

WHY NOT THIS? (TRADEOFFS)

Every model here leans towards popular films (median 216–346 watchers); Day 159 looks at that bias.

4. Stars as confidence

DEFINITION — start here, no prior knowledge assumed

The stars need not be thrown away: they can set the confidence instead. A 5-star film gets $c = 1 + 5\alpha$, a 1-star film $1 + \alpha$. Since stars average about 3.5, the scale was tried from $\alpha/20$ to α per star, keeping the factor count and penalty chosen above, and chosen on validation precision on watched films.

REAL-WORLD SCENARIO

A purchase that was later reviewed with five stars counts for more than one that was returned.

ACTUAL OUTPUT

```
confidence = 1 + alpha x stars instead of 1 + alpha (50 factors, penalty 30); validation precision@10 watched / liked:
  alpha 0.15 per star: 0.1212 / 0.0935
  alpha 0.3 per star: 0.1313 / 0.1040
  alpha 0.6 per star: 0.1386 / 0.1100
  alpha 1 per star: 0.1360 / 0.1080
  alpha 1.5 per star: 0.1299 / 0.1027
  alpha 3 per star: 0.1235 / 0.0950
  binary, alpha 3: 0.1374 / 0.1061
test, stars as confidence (alpha 0.6 per star): watched 0.3703, liked 0.2741
Actual output from this lesson's run.
```

What it means

Validation chose alpha 0.6 per star, so a 5-star film counts 1 + 3 = 4, the same as every watched film in the binary model. **First draft:** the first run tried $\alpha/5$ to α per star and chose $\alpha/5 = 0.6$, at the edge, so $\alpha/10$ and $\alpha/20$ were added; both did worse.

The gain is small: test precision on watched films 0.3703 against the binary model's 0.3672, and on liked films 0.2741 against 0.2653. Whether a difference this size survives a different split was not tested here; Day 159 asks how sure such a difference is.

WHY CHOOSE THIS?

Uses the extra signal where it exists, at no cost; liked films gain most.

WHY NOT THIS? (TRADEOFFS)

Most implicit data has no stars; play counts or watch time need their own mapping to confidence.

5. The other cold start: brand-new users

DEFINITION — start here, no prior knowledge assumed

20% of users (188) are held out entirely; the models are trained on the other 755. Each new user then reveals n = 0, 1, 3, 5, 10 or 20 of their films at random, and the models must find the rest.

Fold-in: with the film factors Y fixed, a new user's factors are one user step of implicit ALS, a k × k solve from the films they watched. No retraining. **Age and gender popularity** recommends what users in the same age band (under 25, 25–34, 35–44, 45+) and gender watched most, from the users' table (load_users); its control gives each new user another new user's group.

REAL-WORLD SCENARIO

A new member of a video club: the shop can guess from their age, or wait for their first few rentals.

MATHEMATICS

$$\textit{fold-in: } x_{new} = (Y^T Y + Y_w{}^T \, diag(c - 1)\, Y_w + \lambda I)^{-1} \, Y_w{}^T c_w, \quad Y \text{ unchanged}$$

IMPLEMENTATION

```
def fold_in(Y, items, conf, lam):
    k = Y.shape[1]; Bj = Y[items]
    G = Y.T @ Y + lam * np.eye(k)
    return np.linalg.solve(G + (Bj * (conf - 1)[:, None]).T @ Bj, Bj.T @ conf)
```

ACTUAL OUTPUT

```
188 new users held out entirely (every one has at least 20 films); models trained on the other 755 users
recall@10 for new users, by how many of their films are known (popularity breaks ties when nothing is known):
                  films known       0       1       3       5      10      20
                   popularity  0.0791  0.0786  0.0787  0.0804  0.0824  0.0925
     age and gender popularity  0.0797  0.0799  0.0806  0.0821  0.0849  0.0943
      control: groups shuffled  0.0719  0.0726  0.0726  0.0750  0.0784  0.0886
                  co-occurrence  0.0791  0.0820  0.1031  0.1119  0.1252  0.1418
          implicit ALS fold-in  0.0791  0.0803  0.1005  0.1115  0.1351  0.1584
Actual output from this lesson's run.
```

What it means

With nothing known, every personal model falls back on popularity (0.0791). Age and gender added almost nothing (0.0797). The control shows the group is not meaningless: a wrong group scored 0.0719, worse than plain popularity. A likely reason, not tested here, for the tiny gain is that each of the 8 groups has far fewer users than the whole, so its popularity list is noisier.

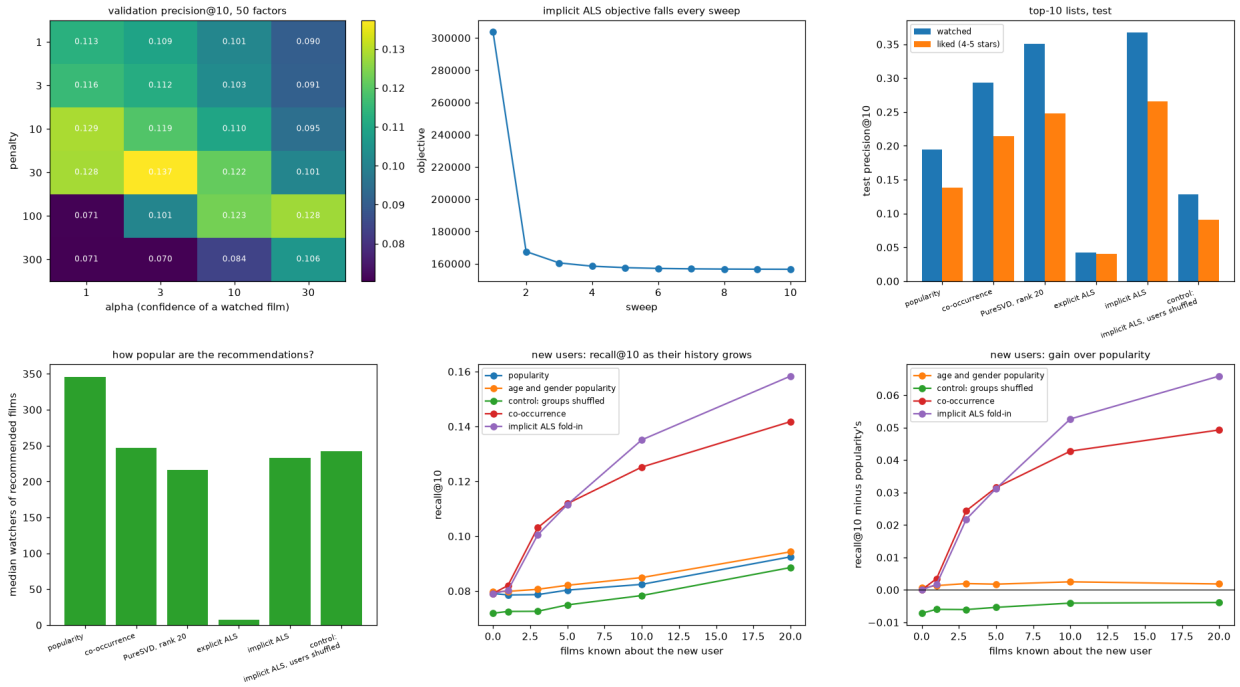
A few films change everything. With 3 known, co-occurrence reached 0.1031 and fold-in 0.1005; the simple neighbour method is level with or slightly ahead of the factors at 1–5 films. From 10 films fold-in pulled ahead (0.1351 against 0.1252), and at 20 it reached 0.1584 against popularity's 0.0925.

WHY CHOOSE THIS?

Fold-in serves new users instantly from the trained model; asking for a handful of films at sign-up pays off fast.

WHY NOT THIS? (TRADEOFFS)

With 0–1 films nothing personal works; demographics helped little here and raise privacy and fairness questions.



Top: validation precision@10 over penalty and α at 50 factors; the implicit ALS objective per sweep; test precision@10 on watched and liked films. Bottom: how popular each model's recommendations are; new users' recall@10 as more of their films are known; and its gain over popularity.

Interview preparation — technical questions

Q: What is the difference between explicit and implicit feedback?

A: Explicit is stated (stars); implicit is observed (watches, clicks). Implicit has no negatives: an empty cell may be dislike or never seen.

Q: How does implicit ALS handle the missing cells?

A: It fits every cell: 1 for watched with confidence $1 + \alpha$, 0 for empty with confidence 1. $Y^T Y$ is shared, so each user step only adds terms for the films they watched.

Q: Why did the explicit model rank so badly?

A: It was trained only on rated cells, so it never learned which films people choose; its lists had a median of 7 watchers and precision@10 0.0421.

Q: How do you recommend to a new user without retraining?

A: Fold-in: keep the film factors and solve one user step from the films they have watched (one $k \times k$ solve).

Q: How would you check a top-10 gain is personal?

A: Shuffle users' factors between users: here precision fell from 0.3672 to 0.1288, below popularity.

Interview preparation — theoretical questions

Q: Why give empty cells any weight at all?

A: Without them every film could score 1 for everyone; the weak 0s are what make the model prefer some films over others.

Q: Recall or precision?

A: Precision@10 asks how good the list is; recall@10 asks how much of what the user will watch it finds, fairer when users watch very different numbers of films.

Q: Is popularity a good baseline?

A: A strong one: 0.1948 here, and it beat every rating model of the week on top-10 lists. Any personal model must beat it.

Q: Do demographics solve user cold start?

A: Not here: age and gender lifted recall@10 from 0.0791 to 0.0797. A handful of the user's own choices did far more.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Treating the data as watched / not watched, and fitting every cell with a confidence weight, changed the week's top-10 results: implicit ALS beat popularity on both watched and liked films, where every star-rating model had lost to it. Day 155's model was not wrong; it answered a different question. For new users, a handful of their own choices was worth far more than their age and gender, and the trained film factors served them with one solve.

- Test precision@10 on watched films: popularity 0.1948, co-occurrence 0.2930, PureSVD 0.3507, implicit ALS 0.3672.
- On liked films: popularity 0.1381, implicit ALS 0.2653, with stars as confidence 0.2741.
- Explicit ALS 0.0421; the shuffled control 0.1288.
- New users with 20 films known: popularity 0.0925, co-occurrence 0.1418, fold-in 0.1584.

Suggested watch order, if starting from zero

- 1. Build a Spotify-Like Music Recommender System in Python (Valerio Velardo - The Sound of AI) — a Python walkthrough building a music-artist recommender with collaborative filtering and ALS.
- 2. Mitigating the challenges of cold start in TensorFlow Recommenders (TensorFlow) — an overview of common approaches to cold start in TensorFlow Recommenders.

Day 159 preview

- **Evaluating recommenders.** Precision@10 has been the week's only ranking measure. Day 159 builds precision@k, recall@k, MAP and NDCG from scratch, asks what each one rewards, and looks at popularity bias and how sure a difference between two models really is.